

Technical Design Specification

Architecture • AI Modeling • Database • Disaster Recovery

Product:	DBDOME dbanalytics service (on-prem database security & monitoring)
Document:	Technical Design Specification (TDS)
Version:	1.0
Date:	30 September 2026
Classification:	Confidential – Internal / NDA
Status:	Issued for review

Document Control

Field	Detail
Title	DBDOME dbanalytics Service — Technical Design Specification
Scope	On-prem DBDOME service: collection, analysis, alerting, web/API, Grafana UI, database
Audience	Engineering, architecture review, security review, pre-sales technical
Related	docs/LOW_LEVEL_DESIGN.md; DBDOME_FUNCTIONALITIES.md; DBDOME_dbanalytics_service_SPEC.md
Version	1.0 — issued for review
Date	30 September 2026

Revision history

Ver	Date	Author	Notes
1.0	2026-09-30	Engineering	Initial issue: architecture, AI modeling, database design, disaster recovery (incl. PG_HOST multi-host failover).

Contents

- 1. Introduction & Scope
- 2. System Architecture
- 3. Component & Module Design
- 4. Collection, Alerting & Response
- 5. Database Design
- 6. AI Modeling
- 7. Database Disaster Recovery
- 8. Security, Secrets & IP Protection
- 9. Web/API & Grafana
- 10. Build, Deploy & Operations
- A. Glossary

1. Introduction & Scope

DBDOME (module `dbdome_dbanalytics_service`) is an on-premise database security and monitoring product. A single Python codebase is frozen into one executable that runs as two Windows services against a bundled PostgreSQL, with **Grafana as the user-facing UI**. It monitors relational database fleets — SQL Server, Oracle, PostgreSQL, MySQL, MariaDB, ClickHouse and Informix — for performance, health and security, raises incident-aware alerts, forwards to SIEM, runs governance/compliance scanners, and generates scheduled PDF/CSV reports.

This Technical Design Specification describes the internal design: runtime and process model, module layout, the collection/alerting pipeline, the database design, the AI modeling layers, the database disaster-recovery design, and the security / secrets / IP protections.

1.1 Distinguishing characteristics

- **Grafana-fronted.** The login surface and dashboards are Grafana (:3000); DBDOME serves JSON APIs and config pages on :8080 that dashboards link out to.
- **Security-forward.** Data Activity Monitoring, SQL-injection and sensitive-column analytics, GRC scanners (SoD, privilege change, DDL audit, TLS enforcement, ransomware guard), tamper-evident hash-chained audit, and active response (session kill).
- **Compliance reporting.** Credential-free JSON report templates including India DPDPA / IRDAI / RBI / CERT-In packs, rendered to PDF+CSV and emailed on a customer cadence.
- **Air-gap capable.** Single frozen bundle; SMTP, reporting and optional local-LLM features operate with no outbound internet.

2. System Architecture

One codebase, one frozen executable invoked with a role flag to run either service; Grafana runs as its own service from the same bin. All state lives in a bundled PostgreSQL (`dbanalytics`).

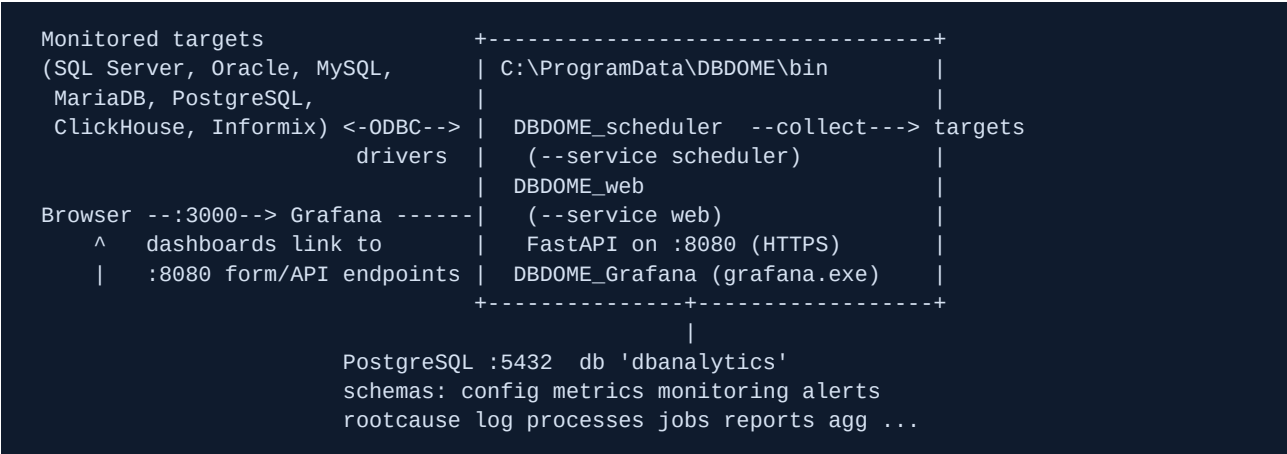


Figure 1 — On-prem runtime topology (two DBDOME services + Grafana).

2.1 Runtime & entry points

Entry point	Role
<code>dbdome_main.py</code>	Bootstrap: starts uvicorn web (optional TLS) and/or the scheduler loop
<code>dbdome_service.py</code>	Windows service wrapper (<code>--service web</code> <code>--service scheduler</code>), NSSM-managed
<code>http_server.py</code>	The FastAPI application (~8.8k lines): all HTTP endpoints

Entry point	Role
job_operation_scheduler.py	APScheduler; 30 s config-watcher reads metrics.registered_processes (is_active toggles jobs live — no restart); max_instances=1, coalesce, SIGTERM-aware

The services have no crash-recovery configured and deploys do not restart them — :8080 must be verified after any overlay.

2.2 Pipeline

Stage	Function
Collection	Per-vendor collectors pull metrics/plans/audit on registered intervals; every path runs the excluded-logins filter.
Analysis	Threshold, custom-metric, threat, SQL-injection, sensitive-column and user-risk analysers; z-score anomaly agent; optional LLM triage.
Alerting	Findings land in alerts.alert_log (partitioned); BEFORE-INSERT suppression; AFTER-INSERT incident upsert.
Response	Email + SIEM fan-out; active response (app-login guard / session kill / threat & ransomware engines); cloud ingest.
Delivery	Grafana dashboards + PWA, scheduled PDF/CSV reports, email, SIEM.

3. Component & Module Design

Module	Contents
collection/	Per-vendor collectors (mssql, oracle, mysql, MariaDB, postgres, clickhouse HTTP :8123, informix); collect_metrics orchestration; monitored_servers_routines; purge/.
analysis/	Metric analysers (generic/custom, SQL-injection, sensitive-column access, threats, user-risk) + LLM agents (anomaly_agent, rootcause_explainer, self_activity_agent) and self_activity_filter.
processes/	Scheduled processes (metrics.registered_processes): GRC scanners (SoD, privilege-change, DDL audit, TLS, vulnerability, cross-border, ransomware guard), data-protection (masking, auto-mask, discovery), app_login_guard, blocker (session kill), internal_health_monitor, gmmr_maintenance, detection-tree builder/executor, retention_engine, hash-chain writer/verifier, compliance_report_generator, threat_response_engine, rbac_provisioning.
alerts/ , siem/	alert_dispatcher (mail/SIEM fan-out; canonical resultset via get_alert_log_resultset_byid), metrics_api_sender (cloud ingest); Wazuh/CrowdStrike/FortiAnalyzer/Rapid + generic CEF/LEEF/RFC5424/JSON senders.
utils/	config_dotenv (connection string), secrets_crypto (Fernet enc:v1:), log4dbexpert (db_write_log), ssl_cert (customer TLS), ldap_settings (Grafana AD).
ReportGenerator/, jobs/, email_utils/	reportlab PDF + Grafana panel capture; scheduled report jobs (PDF+CSV); SMTP senders.
sql_scripts/	Numbered idempotent DB-object catalog NNNN_*.sql, applied in numeric order (tip 7410).

4. Collection, Alerting & Response

```

sync_jobs (30s) - reads metrics.registered_processes (is_active)
  | per interval
  v
process_handler.execute_process -> collection/<vendor>/monitoring_metrics_* -> targets
  |
  |               self_activity_filter (excluded logins)
  |               v
+-----> monitoring.general_metric_metadata_results (gmmr, jsonb)
              v
          analysis/analyse_* (thresholds, threats, SQLi, user-risk)
              v
          alerts.alert_log (partitioned)

```

Figure 2 — Collection through alerting.

- **Suppression:** a BEFORE-INSERT trigger on alert_log drops rows for excluded and whitelisted applicative logins, so no downstream path sees them.
- **Incidents:** an AFTER-INSERT trigger upserts alerts.alert_incidents (open → resolved; auto-resolve configurable; manual resolve via a Grafana-linked form).
- **Fan-out:** alert_dispatcher sends email (canonical detection resultset) and SIEM.
- **Active response:** app_login_guard (watched login from watched program → alert + optional session kill via blocker.py, gated by blocker_dry_run); threat_response_engine; ransomware_guard.
- **Self-monitoring:** internal_health_monitor raises disk/CPU/memory/service-down alerts about the DBDOME host itself.
- **Ingest:** metrics_api_sender posts findings/heartbeats to the dbexpert.ai cloud via organizations_servers links (a NULL api_url silently disables sends).

5. Database Design

All state lives in one bundled PostgreSQL database, dbanalytics, organised into purpose-specific schemas.

Schema	Role
config	global_params (blocker_dry_run, disk_size, grafana_db, local_ip, install identity), mail_config + mail_groups, siem, ldap_settings, report/mail job configs.
metrics	servers (targets; passwords enc:v1:), servers_routines, registered_processes (job registry), app_logins/programs watchlists, exclude_logins.
monitoring	general_metric_metadata_results (gmmr — central results store, partitioned, maintained by gmmr_maintenance SECURITY DEFINER), Grafana v_* views, parse_execution_plan.
rootcause	Detection taxonomy: root causes, detection steps (content encrypted at rest via pgcrypto, per-session key), resolution content, get_alert_log_resultset_byid, v_root_cause_alerts.
alerts	alert_log (two-level partitioning RANGE(entry_date) → HASH(root_cause_id)), suppression triggers, alert_incidents lifecycle, mail alert log.
log / processes / jobs	operation_log + DAM audit views; process bookkeeping; job schedules.
threats / agg / reports / widget / web	Threat analytics, aggregations, generated reports, dashboard widgets, web UI state.

5.1 Roles & object ownership

Two roles: `dbdome_adm` (runtime owner) and `dbdome_mon_usr` (monitoring; member of `adm`). Objects restored from dumps arrive owned by `postgres`; numbered scripts re-assert ownership with guarded `ALTER ... OWNER TO dbdome_adm` blocks. A phased RBAC hardening adds `dbdome_engine` / `dbdome_grafana_ro` roles separating the engine from the read-only Grafana datasource.

5.2 Object deployment (numbered scripts)

DB objects are numbered idempotent scripts applied in numeric order (header rationale, `CREATE OR REPLACE / IF NOT EXISTS`, guarded owner re-assert). A pure-SQL change is applied to the live DB, copied into the four `_internal/sql_scripts` trees, and baked into the golden install dump so fresh installs converge — no rebuild required.

5.3 Partitioning & data lifecycle

`gmmr` and `alert_log` are partitioned; `gmmr_maintenance` (SECURITY DEFINER) upkeep partitions and the retention engine / purgers prune aged data. These are load-bearing — a stalled maintenance job presents as “missing data”, not an error.

5.4 HA-ready connection layer

Access is centralised in `get_connection_string()`. `PG_HOST` accepts a comma-separated failover list (primary, alternate) expanded into a `libpq` multi-host DSN with `target_session_attrs=read-write` and a bounded `connect_timeout`; a single host behaves exactly as before. This is the client-side element of the DR design in §7.

6. AI Modeling

DBDOME's AI spans deterministic statistical detection, ML embeddings for semantic matching, and optional LLM agents for triage and explanation. A consistent principle applies: the **deterministic detection engine drives alerts**; language models only triage and explain, so a model error can never manufacture or suppress a security finding.

6.1 Statistical anomaly agent (z-score)

`analysis/anomaly_agent.py` flags anomalous transactions captured in `gmmr` on three axes — CPU (`cpu_time`), DURATION (`total_elapsed_time`) and AFTER_HOURS (activity outside business hours / weekends) — by **z-score**. The detector is deterministic; each anomaly is then:

- **MATCHED** to the closest existing root cause (AFTER_HOURS → SEC-SQL-ACC-010-RC07; CPU → HLTH-SQL-AD-003-RC04; DURATION → HLTH-SQL-AD-001-RC02; precedence AFTER_HOURS > CPU > DURATION, per-vendor with a fallback),
- **STORED** as a finding row in `gmmr` (matched root cause + the anomalous transaction + a human-readable description),
- **ALERTED** into `alerts.alert_log` (one alert per server+root-cause per hour, matching the collectors' cadence).

6.2 ML embeddings for semantic matching

A sentence-transformer model (all-MiniLM-L6-v2, path configured via the LLM key in `.env`; PyTorch bundled) provides embeddings for semantic matching of findings/queries to taxonomy content — enabling nearest-root-cause selection beyond exact string matching, and running fully offline.

6.3 LLM agents (headless, read-only)

Three optional agents use the Claude Code CLI in headless mode (`c laude -p`) rather than an API-key SDK, with bounded, read-only prompts:

Agent	Purpose
<code>anomaly_agent --explain</code>	Optional Claude triage layered on the deterministic z-score detector
<code>rootcause_explainer</code>	Natural-language explanation of a detected root cause and its evidence
<code>self_activity_agent</code>	Reviews DBDOME's own activity; <code>self_activity_filter</code> enforces excluded logins

Because the agents shell out to `c laude -p`, no `anthropic` pip package or embedded API key is required in the frozen build; an air-gapped site simply does not enable the LLM-triage features while deterministic detection, embeddings and reporting continue to function.

6.4 Threat & user-risk analytics

Beyond single findings, `analyse_threats`, `analyse_user_risk` and the SQL-injection / sensitive-column analysers populate threat and risk models feeding the security dashboards, GRC scanners and SIEM.

Design note: alerts, incidents, reports and SIEM events are produced by the deterministic analysers and the signed detection taxonomy. The z-score agent is deterministic; the LLM agents are advisory triage/explanation only.

7. Database Disaster Recovery

DR for the appliance's own PostgreSQL database (`dbanalytics`), split into what ships today and the recommended production architecture.

7.1 What ships today

Capability	Mechanism
Golden install dump	Schema + taxonomy ship as a <code>pg_dump</code> custom-format artifact (<code>dbanalytics_install.backup</code> , <code>-Fc -Z9 --no-owner --no-acl</code>); fresh installs restore it, then numbered scripts re-assert ownership.
Deterministic schema	Numbered idempotent <code>sql_scripts</code> applied in order; a pure-SQL change is baked into the golden dump so installs converge without a rebuild.
Secret survivability	Secrets are Fernet-encrypted with <code>DBDOME_SECRET_KEY</code> kept in <code>.env</code> OUTSIDE the database — a stolen dump/backup is useless without the key (it must be backed up separately and securely).
Data lifecycle	<code>gmmr</code> / <code>alert_log</code> partitioning + retention engine + purgers keep the working set bounded and restore-friendly.
Tamper-evident audit	Hash-chain writer/verifier make the audit trail tamper-evident, so post-incident integrity of the record can be proven.
HA-ready client DSN	<code>PG_HOST=primary,alternate</code> fails over via <code>libpq</code> multi-host (\$5.4) — shipped this cycle.

The shipped mechanisms provide reliable rebuild-from-artifact recovery plus client-side failover. They do not by themselves provide point-in-time recovery of accumulated findings or automatic standby promotion — the recommended additions below.

7.2 Recommended DR architecture

- **Scheduled logical backups (RPO hours).** Daily/hourly `pg_dump` of `dbanalytics` to off-host, retention-managed, encrypted storage. Back up `DBDOME_SECRET_KEY` separately, or encrypted secrets cannot be decrypted after restore.

- **Continuous archiving / PITR (RPO minutes).** WAL archiving + periodic pg_basebackup for point-in-time recovery — the right posture when losing hours of security findings is unacceptable.
- **Streaming replication + client failover (RTO minutes).** A hot standby kept current by streaming replication; on primary loss the standby is promoted and DBDOME reconnects automatically because its DSN already lists PG_HOST=primary, standby with target_session_attrs=read-write — libpq skips the read-only node until promotion.
- **Grafana state.** grafana.db (SQLite) and conf / live on the appliance filesystem, not in PostgreSQL — include them in the host backup so dashboards, users and provisioning survive a rebuild.
- **Rehearsed restore.** Periodic drills restoring into an isolated environment, validating schema, detection-content integrity and a sample of findings/incidents.

7.3 RPO / RTO summary

Tier	Technique	RPO	RTO	Notes
Baseline	Golden dump + numbered scripts	n/a (seed)	Minutes	Rebuild schema + taxonomy
Standard	Scheduled pg_dump off-host (+ key + grafana.db)	≤ 24 h	Tens of min	Whole-DB incl. findings
Enhanced	WAL archiving + PITR	Minutes	Tens of min	Point-in-time recovery
High availability	Streaming replica + client failover	Seconds	Minutes (auto)	Uses PG_HOST multi-host DSN

Standard suits most on-prem sites; Enhanced / High availability for regulated estates where findings are evidence. All tiers reuse the bundled PostgreSQL and must include DBDOME_SECRET_KEY and the Grafana state in the backup set.

8. Security, Secrets & IP Protection

Layer	Mechanism
App-layer secrets	secrets_crypto: Fernet, enc:v1: prefix, for secrets the service must USE (target DB passwords, SMTP, LDAP bind). Key DBDOME_SECRET_KEY in .env, outside the DB; encrypt-on-write, decrypt-on-read; legacy plaintext passthrough.
Detection content	rootcause detection-step logic encrypted at rest via pgcrypto with a per-session key delivered through libpq options (IP-protection Phase 1).
RBAC	dbdome_adm / dbdome_mon_usr runtime roles; phased dbdome_engine / dbdome_grafana_ro least-privilege split.
Transport	FastAPI on :8080 over HTTPS (self-signed by default; customer cert uploadable at /ssl_certificate, bound at startup). Grafana on :3000 with optional LDAP/AD.
Tamper-evidence	Hash-chain writer/verifier over the audit trail; DAM audit views in the log schema.
Active defence	app_login_guard + blocker (session kill, blocker_dry_run gate), threat_response_engine, ransomware_guard; SIEM forwarding of security events.

9. Web/API & Grafana

The web service (uvicorn, :8080) serves small dark-theme HTML config pages and /api/* JSON that Grafana dashboards link to — mail configuration, TLS certificate, LDAP, app-login guard, RBAC provisioning, data-protection actions, panel print/email, and the incident-resolve form. DB access via

`get_connection_string()`; everything logged through `db_write_log`.

- **Grafana** runs as its own service from the same bin; provisioning paths must be absolute (service cwd is bin); `grafana.db` under `ProgramData/data`.
- **Startup IP rewrite**: the web service rewrites dashboard host references in `grafana.db` to the machine address — so importing `http_server` on a live box mutates `grafana.db` (never do so casually).
- **Panel export**: print/email-panel endpoints render Grafana panels to PDF/CSV and emit UTC timestamptz so `$__timeFrom()` filters resolve.
- **Session timeout**: Grafana `[auth]` lifetimes configured in `conf/custom.ini`.

10. Build, Deploy & Operations

- **Build**: PyInstaller with the `venv314` interpreter (carries `reportlab/matplotlib/scipy/ldap3`); a good exe is ~36-39 MB — a ~18 MB bundle means the wrong interpreter. Two Analysis blocks → `dbdome_dbanalytics.exe` + `dbdome_service.exe` + `shared_internal`.
- **Deploy (live)**: stop web + scheduler → wait for exit → back up exes, rename `_internal` → robocopy new `_internal` → RESTORE `_internal/.env` from backup (top-level `bin/.env` untouched) → copy exes → start services → health-check :8080.
- **Media**: overlay the same payload into `C:/installs/dbdome_update` and `dbdome_setup` (no services there), plus the refreshed install dump.
- **Ubuntu appliance**: runs from source under `/opt/dbdome` — deploy = scp changed `.py` + `systemctl` restart.
- **Gotchas**: services have no auto-restart; `ORG_IP` must equal the machine LAN IP; create firewall rules with `profile=any` (a Domain/Private rule on a Public NIC serves loopback only).

Appendix A. Glossary

Term	Meaning
<code>gmmr</code>	<code>monitoring.general_metric_metadata_results</code> — the central partitioned results store.
Root cause / detection step	Named underlying problem; detection steps are signed SQL probes, encrypted at rest.
<code>alert_log</code>	Partitioned alert store (RANGE by date → HASH by root cause) with suppression + incident triggers.
Incident	<code>alerts.alert_incidents</code> open→resolved lifecycle rolled up from co-firing alerts.
Excluded-logins filter	<code>self_activity_filter</code> — removes excluded/whitelisted logins in every collector.
<code>blocker_dry_run</code>	Global gate; when true, active response logs a kill instead of executing it.
<code>secrets_crypto</code> / <code>enc:v1:</code>	Fernet app-layer secret encryption; key in <code>.env</code> , outside the DB.
<code>PG_HOST</code> multi-host	Comma-separated primary,alternate failover expanded to a libpq multi-host DSN.
<code>DBDOME_SECRET_KEY</code>	Fernet key in <code>.env</code> ; must be backed up separately or restored secrets are unreadable.
PITR / RPO / RTO	Point-in-time recovery; Recovery Point / Time Objective.
GRC scanners	Governance/Risk/Compliance processes: SoD, privilege change, DDL audit, TLS, ransomware guard.
<code>claude -p</code>	Headless Claude Code CLI invocation used by the read-only LLM agents.